

=== CONFIGURACION GNB (1779567542) ===

```
# gnb_v2x.yml
#
# This configuration file sets up the srsRAN Project gNB for V2X communications
# using ZeroMQ as the RF frontend, connecting to an Open5GS core.
#
# To run the srsRAN Project gNB with this config, use the following command:
#   sudo ./gnb -c gnb_v2x.yml

cu_cp:
  amf:
    addr: 10.53.1.2          # The address or hostname of the AMF (Open5GS).
    port: 38412              # Standard N2 port for AMF.
    bind_addr: 10.53.1.3     # A local IP that the gNB binds to for traffic from the AMF.
    supported_tracking_areas:
      - tac: 7               # Tracking Area Code (TAC) - must match core configuration.
      plmn_list:
        - plmn: "31501"      # PLMN broadcasted by the gNB (MCC 315, MNC 010).
          tai_slice_support_list:
            - sst: 4          # Slice/Service Type (SST) for V2X services.
                                # From 3GPP TS 23.501 v16.6.0, Table 5.15.2.2-1, Page 199.
    inactivity_timer: 7200   # Sets the UE/PDU Session/DRB inactivity timer to 7200 seconds.

ru_sdr:
  device_driver: zmq         # The RF driver name, using ZeroMQ for virtual RF.
  device_args: tx_port=tcp://127.0.0.1:2000,rx_port=tcp://127.0.0.1:2001,base_srate=23.04e6 # ZMQ ports for gNB (tx:2000,rx:2001)
  srate: 23.04              # RF sample rate in Msps, adjusted for 20 MHz bandwidth.
  tx_gain: 75               # Transmit gain of the RF.
  rx_gain: 75               # Receive gain of the RF.

cell_cfg:
  dl_arfcn: 620000          # ARFCN of the downlink carrier (center frequency).
                              # For Band 78, ARFCN 620000 is within the valid range (620000-673333).
  band: 78                  # The NR band, n78 (3.3-4.2 GHz).
  channel_bandwidth_MHz: 20 # Bandwidth in MHz. Number of PRBs will be automatically derived.
  common_scs: 15            # Subcarrier spacing in kHz used for data.
  plmn: "31501"            # PLMN broadcasted by the gNB (MCC 315, MNC 010).
  tac: 7                    # Tracking area code (needs to match the core configuration).
  pdccch:
    common:
      ss0_index: 0          # Set search space zero index to match srsUE capabilities.
      coresets0_index: 12   # Set search CORESET Zero index to match srsUE capabilities for 20MHz.
    dedicated:
      ss2_type: common      # Search Space type, has to be set to common.
      dci_format_0_1_and_1_1: false # Set correct DCI format (fallback).
  prach:
    prach_config_index: 1   # Sets PRACH config to match what is expected by srsUE.
  pdsch:
    mcs_table: qam64        # Sets PDSCH MCS to 64 QAM.
  pusch:
    mcs_table: qam64        # Sets PUSCH MCS to 64 QAM.

log:
  filename: /tmp/gnb.log    # Path of the log file.
  all_level: info           # Logging level applied to all layers.
  hex_max_size: 0

pcap:
  mac_enable: false         # Set to true to enable MAC-layer PCAPs.
  mac_filename: /tmp/gnb_mac.pcap # Path where the MAC PCAP is stored.
  ngap_enable: false        # Set to true to enable NGAP PCAPs.
  ngap_filename: /tmp/gnb_ngap.pcap # Path where the NGAP PCAP is stored.
  e2ap_enable: true         # Set to true to enable E2AP PCAPs.
  e2ap_du_filename: /tmp/gnb_du_e2ap.pcap # Path where the DU E2AP PCAP is stored.
  e2ap_cu_cp_filename: /tmp/gnb_cu_cp_e2ap.pcap # Path where the CU-CP E2AP PCAP is stored.
```

e2ap_cu_up_filename: /tmp/gnb_cu_up_e2ap.pcap # Path where the CU-UP E2AP PCAP is stored.

metrics:

layers:

enable_rlc: true # Enable RLC metrics.

enable_sched: true # Enable DU scheduler metrics.

periodicity:

du_report_period: 1000 # Set DU statistics report period to 1s.

=== CONFIGURACION UE (1779567542) ===

```
# ue_v2x.conf
#
# This configuration file sets up the srsRAN Project UE for V2X communications
# using ZeroMQ as the RF frontend, connecting to a gNB.
#
# To run the srsRAN Project UE with this config, use the following command:
#   sudo ./srsue -c ue_v2x.conf

[rf]
freq_offset = 0
tx_gain = 50
rx_gain = 40
srate = 23.04e6          # RF sample rate in Hz, matching gNB's 20MHz bandwidth.
nof_antennas = 1

device_name = zmq          # RF driver name, using ZeroMQ for virtual RF.
device_args = tx_port=tcp://127.0.0.1:2001,rx_port=tcp://127.0.0.1:2000,base_srate=23.04e6 # ZMQ ports for UE (tx:2001,rx:2000)

[rat.eutra]
dl_earfcn = 2850          # Not used for NR, but kept for template structure.
nof_carriers = 0

[rat.nr]
bands = 78                # NR band, n78, matching gNB.
nof_carriers = 1

max_nof_prb = 106         # Max number of PRBs for 20MHz bandwidth.
nof_prb = 106             # Number of PRBs for 20MHz bandwidth.

[pcap]
enable = none
mac_filename = /tmp/ue_mac.pcap
mac_nr_filename = /tmp/ue_mac_nr.pcap
nas_filename = /tmp/ue_nas.pcap

[log]
all_level = info
phy_lib_level = none
all_hex_limit = 32
filename = /tmp/ue.log
file_max_size = -1

[usim]
mode = soft               # USIM mode.
algo = milenage           # USIM algorithm.
opc = 63BFA50EE6523365FF14C1F45F88737D # Operator Code (OPc).
k = 00112233445566778899aabbccddeeff # Subscriber Key (K).
imsi = 315010000000001    # International Mobile Subscriber Identity (IMSI).
                          # MCC: 315, MNC: 010, MSIN: 000000001.

[rrc]
release = 15              # 3GPP Release 15.
ue_category = 4           # UE category.

[nas]
apn = srsapn              # Access Point Name.
apn_protocol = ipv4        # APN protocol.

[gw]
netns = ue1               # Network namespace for the UE.
ip_devname = tun_srsue     # Tunnel device name.
ip_netmask = 255.255.255.0 # IP netmask.

[gui]
```

enable = false

=== DOCKER COMPOSE (1779567542) ===

```
# docker-compose.yml
#
# Copyright 2021-2026 Software Radio Systems Limited
#
# This file is part of srsRAN
#
# srsRAN is free software: you can redistribute it and/or modify
# it under the terms of the GNU Affero General Public License as
# published by the Free Software Foundation, either version 3 of
# the License, or (at your option) any later version.
#
# srsRAN is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
# GNU Affero General Public License for more details.
#
# A copy of the GNU Affero General Public License can be found in
# the LICENSE file in the top-level directory of this distribution
# and at http://www.gnu.org/licenses/.
#

services:
  5gc:
    container_name: open5gs_5gc
    build:
      context: open5gs
      target: open5gs
      args:
        OS_VERSION: "22.04"
        OPEN5GS_VERSION: "v2.7.0"
    env_file:
      - ${OPEN_5GS_ENV_FILE:-open5gs/open5gs.env}
    privileged: true
    ports:
      - "9898:9999/tcp"
      # Uncomment port to use the 5gc from outside the docker network
      # - "38412:38412/sctp"
      # - "2152:2152/udp"
    command: 5gc -c open5gs-5gc.yml
    healthcheck:
      test: [ "CMD-SHELL", "nc -z 127.0.0.20 7777" ]
      interval: 3s
      timeout: 1s
      retries: 60
    networks:
      ran:
        ipv4_address: 10.53.1.2 # Explicitly assign AMF IP as requested

  gnb:
    container_name: srsran_gnb
    # Build info
    image: srsran/gnb
    build:
      context: ..
      dockerfile: docker/Dockerfile
      args:
        OS_VERSION: "24.04"
      # privileged mode is required only for accessing usb devices
      privileged: true
      # Extra capabilities always required
      cap_add:
        - SYS_NICE
        - CAP_SYS_PTRACE
    volumes:
```

```

# Access USB to use some SDRs
- /dev/bus/usb/:/dev/bus/usb/
# Sharing images between the host and the pod.
# It's also possible to download the images inside the pod
- /usr/share/uhd/images:/usr/share/uhd/images
# Save logs and more into gnb-storage
- gnb-storage:/tmp
# It creates a file/folder into /config_name inside the container
# Its content would be the value of the file used to create the config
configs:
  - gnb_config.yml
  - gnb_compose_config.yml
# Customize your desired network mode.
# current network configuration creates a private network with both containers attached
# An alternative would be `network: host`. That would expose your host network into the container. It's the easiest
networks:
  ran:
    ipv4_address: 10.53.1.3 # Explicitly assign gNB IP as requested
  metrics:
    ipv4_address: 172.19.1.3
# Start GNB container after 5gc is up and running
depends_on:
  5gc:
    condition: service_healthy
# Command to run into the final container
command: gnb -c /gnb_config.yml -c /gnb_compose_config.yml

configs:
  gnb_config.yml:
    file: ${GNB_CONFIG_PATH:-../configs/gnb_v2x.yml} # Path to the generated gNB config file
  gnb_compose_config.yml:
    content: |
      cu_cp:
        amf:
          addr: 10.53.1.2 # AMF IP, matching the service definition
          bind_addr: 10.53.1.3 # gNB IP, matching the service definition
        metrics:
          autostart_stdout_metrics: true
          enable_json: true
        remote_control:
          bind_addr: 0.0.0.0
          enabled: true

volumes:
  gnb-storage:

networks:
  ran:
    ipam:
      driver: default
      config:
        - subnet: 10.53.1.0/24
  metrics:
    ipam:
      driver: default
      config:
        - subnet: 172.19.1.0/24

# Tutorial: ZeroMQ-based Setup for srsRAN Project (adapted for Docker)
#
# This tutorial explains how to set up and test connectivity for srsRAN Project
# gNB and UE using ZeroMQ for virtual RF, with Open5GS as the 5G Core,
# leveraging the Docker setup defined above.
#
# Prerequisites:
# - Docker and Docker Compose installed.
# - srsRAN Project source code cloned (assuming docker-compose.yml is in srsran_project/docker).
# - The generated 'gnb_v2x.yml' should be placed in 'srsran_project/configs/'.

```

```

# - The generated 'ue_v2x.conf' can be placed anywhere, as srsUE will be run outside Docker.
#
# Step 1: Prepare the srsRAN Project environment
# -----
# Ensure your srsRAN Project build supports ZeroMQ. If building from source,
# navigate to your srsRAN Project root directory and build it:
#
#   cd srsran_project
#   mkdir build && cd build
#   cmake -DENABLE_ZMQ=ON ..
#   make -j$(nproc)
#   sudo make install
#   sudo srsran_install_configs.sh
#
# Step 2: Place Configuration Files
# -----
# - Save the generated gNB configuration as `srsran_project/configs/gnb_v2x.yml`.
# - Save the generated UE configuration as `ue_v2x.conf` in a convenient location.
#
# Step 3: Start the 5G Core and gNB using Docker Compose
# -----
# Navigate to the directory containing your `docker-compose.yml` (e.g., `srsran_project/docker`).
#
#   cd srsran_project/docker
#   docker-compose up -d 5gc gnb
#
# This command will:
# - Build and start the Open5GS 5GC container (`open5gs_5gc`).
# - Build and start the srsRAN gNB container (`srsran_gnb`).
# - The gNB container will wait for the 5GC to be healthy before starting.
# - The gNB will load `gnb_v2x.yml` and the compose-specific overrides.
#
# You can check the logs of the running containers:
#   docker-compose logs -f 5gc
#   docker-compose logs -f gnb
#
# Step 4: Run the srsUE
# -----
# Open a new terminal. The srsUE will run directly on your host machine,
# connecting to the gNB's ZMQ interface.
#
#   sudo srsue -c /path/to/your/ue_v2x.conf
#
# Replace `/path/to/your/ue_v2x.conf` with the actual path to your UE configuration file.
#
# Step 5: Verify Connectivity
# -----
# Once the srsUE starts, observe its logs and the gNB logs (from the Docker terminal).
#
# Expected log messages:
# - In gNB logs: Look for "RRC Setup Complete", "PDU Session Establishment Request", "PDU Session Establishment Accept"
# - In UE logs: Look for "RRC Connected", "PDU Session Established".
#
# After successful connection, the UE should have an IP address assigned. You can verify this
# and test internet connectivity from the UE's network namespace:
#
#   sudo ip netns exec ue1 ip a
#   sudo ip netns exec ue1 ping -I tun_srsue 8.8.8.8
#
# This will send ICMP requests from the UE's tunnel interface (`tun_srsue`) within its
# dedicated network namespace (`ue1`) to an external IP (e.g., Google's DNS).
# Successful pings indicate end-to-end connectivity through the 5G network.
#
# Step 6: Clean up (Optional)
# -----
# To stop and remove the Docker containers:
#   cd srsran_project/docker
#   docker-compose down

```

#

To stop the srsUE, press Ctrl+C in its terminal.